10. 장애 처리와 복구

10.1 장애 처리 개요

Narwhal 시스템은 비잔틴 장애 허용 시스템으로, 다양한 유형의 장애에 대응할 수 있는 강력한 복구 메커니즘을 제공합니다.

10.1.1 장애 유형 분류

- **비잔틴 장애**: 악의적이거나 임의적인 노드 행동
- **크래시 장애**: 노드의 갑작스러운 중단
- **네트워크 장애**: 통신 연결 문제
- **성능 장애**: 느린 응답이나 처리 지연

10.1.2 장애 허용 수준

```
pub struct FaultToleranceConfig {
pub total_nodes: usize,
pub byzantine_fault_threshold: usize, // f = (n-1)/3
pub crash_fault_threshold: usize, // 2f+1 생존 필요
pub network_partition_threshold: Duration,
pub recovery_timeout: Duration,
}
```

```rust
impl FaultToleranceConfig {
pub fn new(total_nodes: usize) -> Self {
let byzantine_fault_threshold = (total_nodes - 1) / 3;
let crash_fault_threshold = total_nodes - 2 * byzantine_fault_threshold - 1;
FaultToleranceConfig {
total_nodes,
byzantine_fault_threshold,
crash_fault_threshold,
network_partition_threshold: Duration::from_secs(30),
recovery_timeout: Duration::from_secs(60),
}
}
pub fn can_make_progress(&self, active_nodes: usize) -> bool {
active_nodes >= 2 * self.byzantine_fault_threshold + 1
}
}
```

10.2 장애 감지 시스템

10.2.1 노드 상태 모니터링

```rust
pub struct NodeHealthMonitor {
node_states: HashMap<PublicKey, NodeState>,
```

```
health_check_interval: Duration,
failure_threshold: usize,
recovery_threshold: usize,
}

#[derive(Clone, Debug)]
pub struct NodeState {
pub status: NodeStatus,
pub last_seen: Instant,
pub consecutive_failures: usize,
pub response_times: VecDeque<Duration>,
pub reputation_score: f64,
}

#[derive(Clone, Debug, PartialEq)]
pub enum NodeStatus {
Healthy,
Degraded,
Suspected,
Failed,
Recovering,
}
```

```
impl NodeHealthMonitor {
pub  async  fn  monitor_nodes(&mut  self) {
let  mut  interval = tokio::time::interval(self.health_check_interval);
loop {
interval.tick().await;
for (node_id, state) in &mut  self.node_states {
// 헬스 체크 수행
match  self.perform_health_check(node_id).await {
Ok(response_time) => {
self.handle_successful_health_check(state, response_time);
}
Err(error) => {
self.handle_failed_health_check(state, error);
}
}
// 상태 업데이트
self.update_node_status(state);
}
}
}
async  fn  perform_health_check(&self, node_id: &PublicKey) -> Result<Duration,
let  start_time = Instant::now();
// 간단한 ping 메시지 전송
```

```
let ping_message = HealthCheckMessage::Ping {
timestamp: SystemTime::now(),
sender: self.local_node_id.clone(),
};
// 타임아웃과 함께 응답 대기
let response = timeout(
Duration::from_secs(5),
self.send_health_check(node_id, ping_message)
).await??;
match response {
HealthCheckMessage::Pong { timestamp, .. } => {
Ok(start_time.elapsed())
}
_ => Err(HealthCheckError::InvalidResponse),
}
}
fn handle_successful_health_check(&self, state: &mut NodeState, response_time:
state.last_seen = Instant::now();
state.consecutive_failures = 0;
state.response_times.push_back(response_time);
// 응답 시간 히스토리 제한
if state.response_times.len() > 10 {
state.response_times.pop_front();
```

```
}
// 평판 점수 향상
state.reputation_score = (state.reputation_score * 0.9 + 1.0 * 0.1).min(1.0);
}
fn handle_failed_health_check(&self, state: &mut NodeState, error: HealthCheckE
state.consecutive_failures += 1;
// 평판 점수 감소
state.reputation_score = (state.reputation_score * 0.9).max(0.0);
tracing::warn!("Health check failed for node: {:?}", error);
}
fn update_node_status(&self, state: &mut NodeState) {
let time_since_last_seen = state.last_seen.elapsed();
state.status = match state.consecutive_failures {
0 => {
if time_since_last_seen < Duration::from_secs(10) {
NodeStatus::Healthy
} else {
NodeStatus::Degraded
}
}
1..=2 => NodeStatus::Suspected,
_ => {
if time_since_last_seen > Duration::from_secs(60) {
```

```
NodeStatus::Failed
} else {
NodeStatus::Suspected
}
}
};
}
}
```

10.2.2 네트워크 분할 감지

```
pub struct NetworkPartitionDetector {
connectivity_matrix: HashMap<(PublicKey, PublicKey), bool>,
partition_threshold: f64,
detection_window: Duration,
}

impl NetworkPartitionDetector {
pub fn detect_partition(&mut self) -> Option<NetworkPartition> {
let total_connections = self.connectivity_matrix.len();
let active_connections = self.connectivity_matrix.values()
.filter(|&&connected| connected)
.count();
```

```
let connectivity_ratio = active_connections as f64 / total_connections as f64
if connectivity_ratio < self.partition_threshold {
Some(self.analyze_partition_structure())
} else {
None
}
}
fn analyze_partition_structure(&self) -> NetworkPartition {
let mut partitions = Vec::new();
let mut visited = HashSet::new();
// 연결된 컴포넌트 찾기 (DFS)
for (node_a, node_b) in self.connectivity_matrix.keys() {
if !visited.contains(node_a) {
let component = self.find_connected_component(node_a, &mut visited);
if !component.is_empty() {
partitions.push(component);
}
}
}
NetworkPartition {
partitions,
detected_at: SystemTime::now(),
severity: self.calculate_partition_severity(&partitions),
```

```
}
}
fn find_connected_component(&self, start_node: &PublicKey, visited: &mut HashSe
let mut component = Vec::new();
let mut stack = vec![start_node.clone()];
while let Some(node) = stack.pop() {
if visited.contains(&node) {
continue;
}
visited.insert(node.clone());
component.push(node.clone());
// 연결된 이웃 노드들 찾기
for ((node_a, node_b), &connected) in &self.connectivity_matrix {
if connected {
if *node_a == node && !visited.contains(node_b) {
stack.push(node_b.clone());
} else if *node_b == node && !visited.contains(node_a) {
stack.push(node_a.clone());
}
}
}
}
component
```

```
}

}
```

10.3 자동 복구 메커니즘

10.3.1 상태 복구 시스템

```
pub struct StateRecoveryManager {

certificate_store: Arc<CertificateStore>,

consensus_store: Arc<ConsensusStore>,

recovery_strategy: RecoveryStrategy,

sync_manager: SyncManager,

}


impl StateRecoveryManager {

pub async fn recover_from_failure(&mut self) -> Result<(), RecoveryError> {

tracing::info!("Starting state recovery process");

// 1. 로컬 상태 검증

let local_state = self.validate_local_state().await?;

// 2. 피어들로부터 상태 동기화

let peer_states = self.sync_manager.fetch_peer_states().await?;

// 3. 일관된 상태 결정

let consistent_state = self.determine_consistent_state(local_state, peer_states
```

```
// 4. 로컬 상태 복원
self.restore_local_state(consistent_state).await?;
// 5. 복구 검증
self.verify_recovery().await?;
tracing::info!("State recovery completed successfully");
Ok(())
}
async fn validate_local_state(&self) -> Result<LocalState, RecoveryError> {
// 저장된 인증서들의 무결성 검증
let certificates = self.certificate_store.all_certificates()?;
let mut valid_certificates = Vec::new();
let mut corrupted_certificates = Vec::new();
for certificate in certificates {
match self.verify_certificate_integrity(&certificate) {
Ok(()) => valid_certificates.push(certificate),
Err(e) => {
tracing::warn!("Corrupted certificate found: {:?}", e);
corrupted_certificates.push(certificate.digest());
}
}
}
// 컨센서스 상태 검증
let consensus_state = self.consensus_store.read_consensus_state()?;
```

```
let state_valid = self.verify_consensus_state_integrity(&consensus_state)?;
Ok(LocalState {
valid_certificates,
corrupted_certificates,
consensus_state,
state_valid,
last_verified_round: self.find_last_verified_round(&valid_certificates),
})
}
async fn restore_local_state(&mut self, target_state: ConsistentState) -> Res
// 저장소 초기화
self.certificate_store.clear()?;
self.consensus_store.clear()?;
// 인증서 복원 (의존성 순서대로)
let ordered_certificates = self.topological_sort_certificates(&target_state.cer
for certificate in ordered_certificates {
self.certificate_store.write_certificate(certificate)?;
}
// 컨센서스 상태 복원
self.consensus_store.write_consensus_state(
&target_state.last_committed,
&target_state.consensus_index,
&target_state.last_certificate_digest,
```

```
)?;
Ok(())
}
fn topological_sort_certificates(&self, certificates: &[Certificate]) -> Result
let mut sorted = Vec::new();
let mut visited = HashSet::new();
let mut temp_visited = HashSet::new();
// 인증서를 다이제스트로 인덱싱
let cert_map: HashMap<CertificateDigest, &Certificate> = certificates
.iter()
.map(|cert| (cert.digest(), cert))
.collect();
// 각 인증서에 대해 DFS 수행
for certificate in certificates {
if !visited.contains(&certificate.digest()) {
self.dfs_topological_sort(
certificate,
&cert_map,
&mut visited,
&mut temp_visited,
&mut sorted,
)?;
}
```

```
}
sorted.reverse(); // DFS 후처리 순서의 역순이 위상 정렬 순서
Ok(sorted)
}
}
```

10.3.2 점진적 동기화

```
pub struct IncrementalSyncManager {
sync_batch_size: usize,
max_concurrent_syncs: usize,
sync_timeout: Duration,
peer_selector: PeerSelector,
}

impl IncrementalSyncManager {
pub async fn sync_missing_data(&mut self, missing_range: RoundRange) -> Resu
let total_rounds = missing_range.end - missing_range.start + 1;
let batches = (total_rounds + self.sync_batch_size - 1) / self.sync_batch_size;
tracing::info!("Starting incremental sync for {} rounds in {} batches", total_ro
let semaphore = Arc::new(Semaphore::new(self.max_concurrent_syncs));
let mut sync_tasks = Vec::new();
```

```
for batch_index in 0..batches {
let start_round = missing_range.start + batch_index * self.sync_batch_size;
let end_round = std::cmp::min(
start_round + self.sync_batch_size - 1,
missing_range.end
);
let batch_range = RoundRange { start: start_round, end: end_round };
let semaphore_clone = semaphore.clone();
let peer_selector = self.peer_selector.clone();
let task = tokio::spawn(async move {
let _permit = semaphore_clone.acquire().await.unwrap();
Self::sync_batch(batch_range, peer_selector).await
});
sync_tasks.push(task);
}
// 모든 배치 동기화 완료 대기
let results = futures::future::join_all(sync_tasks).await;
// 실패한 배치 확인
let mut failed_batches = Vec::new();
for (index, result) in results.into_iter().enumerate() {
match result {
Ok(Ok(())) => {
tracing::debug!("Batch {} sync completed successfully", index);
```

```
}
Ok(Err(e)) => {
tracing::error!("Batch {} sync failed: {:?}", index, e);
failed_batches.push(index);
}
Err(e) => {
tracing::error!("Batch {} task panicked: {:?}", index, e);
failed_batches.push(index);
}
}
}
if !failed_batches.is_empty() {
return Err(SyncError::PartialSyncFailure {
failed_batches,
total_batches: batches,
});
}
tracing::info!("Incremental sync completed successfully");
Ok(())
}
async fn sync_batch(range: RoundRange, peer_selector: PeerSelector) -> Result<
let peers = peer_selector.select_reliable_peers(3).await?;
for peer in peers {
```

```
match Self::sync_batch_from_peer(range, &peer).await {
Ok(()) => return Ok(()),
Err(e) => {
tracing::warn!("Failed to sync batch from peer {}: {:?}", peer.id, e);
continue;
}
}
}
Err(SyncError::AllPeersFailed { range })
}
async fn sync_batch_from_peer(range: RoundRange, peer: &PeerInfo) -> Result<()
// 피어로부터 해당 라운드 범위의 인증서 요청
let request = SyncRequest {
start_round: range.start,
end_round: range.end,
requester: self.local_node_id.clone(),
};
let response = timeout(
self.sync_timeout,
peer.send_sync_request(request)
).await??;
// 받은 인증서들 검증 및 저장
for certificate in response.certificates {
```

```
// 인증서 유효성 검증
self.verify_certificate(&certificate)?;
// 로컬 저장소에 저장
self.certificate_store.write_certificate(certificate)?;
}
Ok(())
}
}
```

10.4 비잔틴 장애 대응

10.4.1 악의적 행동 감지

```
pub struct ByzantineDetector {
behavior_tracker: HashMap<PublicKey, BehaviorProfile>,
detection_rules: Vec<DetectionRule>,
reputation_system: ReputationSystem,
}

#[derive(Clone, Debug)]
pub struct BehaviorProfile {
pub node_id: PublicKey,
pub message_patterns: MessagePatternAnalyzer,
```

```
pub timing_patterns: TimingPatternAnalyzer,
pub consistency_violations: Vec<ConsistencyViolation>,
pub reputation_score: f64,
}


impl ByzantineDetector {
pub fn analyze_node_behavior(&mut self, node_id: &PublicKey, message: &Primar
let profile = self.behavior_tracker.entry(node_id.clone())
.or_insert_with(|| BehaviorProfile::new(node_id.clone()));
// 메시지 패턴 분석
profile.message_patterns.record_message(message);
// 타이밍 패턴 분석
profile.timing_patterns.record_timing(SystemTime::now());
// 일관성 위반 검사
let violations = self.check_consistency_violations(node_id, message);
profile.consistency_violations.extend(violations);
// 의심스러운 행동 감지
let suspicious_behaviors = self.detect_suspicious_behaviors(profile);
// 평판 점수 업데이트
self.reputation_system.update_reputation(node_id, &suspicious_behaviors);
BehaviorAnalysis {
node_id: node_id.clone(),
suspicious_behaviors,
```

```
confidence_level: self.calculate_confidence_level(profile),

recommended_action: self.determine_recommended_action(profile),

}

}

fn  detect_suspicious_behaviors(&self, profile: &BehaviorProfile) -> Vec<Suspici

let  mut  behaviors = Vec::new();

// 메시지 플러딩 감지

if  profile.message_patterns.messages_per_second() > 1000.0 {

behaviors.push(SuspiciousBehavior::MessageFlooding {

rate: profile.message_patterns.messages_per_second(),

threshold: 1000.0,

});

}

// 일관성 없는 투표 감지

let  inconsistent_votes = profile.consistency_violations.iter()

.filter(|v| matches!(v, ConsistencyViolation::InconsistentVote { .. }))

.count();

if  inconsistent_votes > 5 {

behaviors.push(SuspiciousBehavior::InconsistentVoting {

violation_count: inconsistent_votes,

});

}

// 타이밍 공격 감지
```

```
if profile.timing_patterns.has_suspicious_timing() {
behaviors.push(SuspiciousBehavior::TimingAttack {
pattern: profile.timing_patterns.get_pattern_description(),
});
}
// 잘못된 서명 감지
let invalid_signatures = profile.consistency_violations.iter()
.filter(|v| matches!(v, ConsistencyViolation::InvalidSignature { .. }))
.count();
if invalid_signatures > 0 {
behaviors.push(SuspiciousBehavior::InvalidSignatures {
count: invalid_signatures,
});
}
behaviors
}
fn determine_recommended_action(&self, profile: &BehaviorProfile) -> Recommende
if profile.reputation_score < 0.3 {
RecommendedAction::Isolate {
reason: "Multiple suspicious behaviors detected".to_string(),
duration: Duration::from_hours(1),
}
} else if profile.reputation_score < 0.6 {
```

```
RecommendedAction::Monitor {
reason: "Some suspicious behaviors detected".to_string(),
increased_scrutiny: true,
}
} else {
RecommendedAction::Continue {
reason: "No significant suspicious behavior".to_string(),
}
}
}
}
```

10.4.2 격리 및 복구 메커니즘

```
pub struct NodeIsolationManager {
isolated_nodes: HashMap<PublicKey, IsolationInfo>,
isolation_policies: Vec<IsolationPolicy>,
recovery_manager: RecoveryManager,
}

#[derive(Clone, Debug)]
pub struct IsolationInfo {
pub node_id: PublicKey,
```

```
pub isolation_start: SystemTime,
pub isolation_duration: Duration,
pub isolation_reason: String,
pub evidence: Vec<Evidence>,
pub recovery_attempts: usize,
}

impl NodeIsolationManager {
pub async fn isolate_node(&mut self,
node_id: PublicKey,
reason: String,
evidence: Vec<Evidence>
) -> Result<(), IsolationError> {
tracing::warn!("Isolating node {} due to: {}", node_id, reason);
let isolation_info = IsolationInfo {
node_id: node_id.clone(),
isolation_start: SystemTime::now(),
isolation_duration: self.calculate_isolation_duration(&evidence),
isolation_reason: reason,
evidence,
recovery_attempts: 0,
};
// 노드를 격리 목록에 추가
```

```
self.isolated_nodes.insert(node_id.clone(), isolation_info);

// 네트워크 연결 차단

self.block_network_connections(&node_id).await?;

// 다른 노드들에게 격리 알림

self.broadcast_isolation_notification(&node_id).await?;

// 격리 메트릭 업데이트

self.update_isolation_metrics(&node_id);

Ok(())

}

pub  async  fn  attempt_node_recovery(&mut  self, node_id: &PublicKey) -> Result

let  isolation_info = self.isolated_nodes.get_mut(node_id)

.ok_or(RecoveryError::NodeNotIsolated)?;

isolation_info.recovery_attempts += 1;

tracing::info!("Attempting recovery for node {} (attempt {})",

node_id, isolation_info.recovery_attempts);

// 복구 프로세스 시작

let  recovery_result = self.recovery_manager.attempt_recovery(node_id).await;

match  recovery_result {

Ok(()) => {

// 복구 성공 - 격리 해제

self.lift_isolation(node_id).await?;

tracing::info!("Node {} successfully recovered and reintegrated", node_id);

}
```

```
Err(e) => {
tracing::warn!("Recovery attempt failed for node {}: {:?}", node_id, e);
// 최대 복구 시도 횟수 확인
if  isolation_info.recovery_attempts >= 3 {
// 영구 격리로 전환
self.convert_to_permanent_isolation(node_id).await?;
}
}
}
Ok(())
}
async  fn  lift_isolation(&mut  self, node_id: &PublicKey) -> Result<(), Isolati
// 격리 목록에서 제거
self.isolated_nodes.remove(node_id);
// 네트워크 연결 복원
self.restore_network_connections(node_id).await?;
// 다른 노드들에게 복구 알림
self.broadcast_recovery_notification(node_id).await?;
// 평판 점수 부분 복원
self.reputation_system.partial_reputation_restore(node_id);
Ok(())
}
pub  async  fn  periodic_isolation_review(&mut  self) {
```

```
let current_time = SystemTime::now();
let mut nodes_to_review = Vec::new();
// 격리 기간이 만료된 노드들 찾기
for (node_id, isolation_info) in &self.isolated_nodes {
let isolation_elapsed = current_time.duration_since(isolation_info.isolation_st
.unwrap_or(Duration::ZERO);
if isolation_elapsed >= isolation_info.isolation_duration {
nodes_to_review.push(node_id.clone());
}
}
// 복구 시도
for node_id in nodes_to_review {
if let Err(e) = self.attempt_node_recovery(&node_id).await {
tracing::error!("Failed to recover node {}: {:?}", node_id, e);
}
}
}
}
```

10.5 재해 복구 계획

10.5.1 전체 시스템 복구

```
pub struct DisasterRecoveryManager {
backup_manager: BackupManager,
cluster_coordinator: ClusterCoordinator,
recovery_procedures: Vec<RecoveryProcedure>,
}


impl DisasterRecoveryManager {
pub async fn execute_disaster_recovery(&mut self, disaster_type: DisasterTyp
tracing::critical!("Executing disaster recovery for: {:?}", disaster_type);
match disaster_type {
DisasterType::MajorityNodeFailure => {
self.recover_from_majority_failure().await?;
}
DisasterType::NetworkPartition => {
self.recover_from_network_partition().await?;
}
DisasterType::DataCorruption => {
self.recover_from_data_corruption().await?;
}
DisasterType::SystemCompromise => {
self.recover_from_system_compromise().await?;
}
```

```
}

// 복구 후 시스템 검증

self.verify_system_integrity().await?;

tracing::info!("Disaster recovery completed successfully");

Ok(())

}

async fn recover_from_majority_failure(&mut self) -> Result<(), DisasterRecov

// 1. 생존 노드 식별

let surviving_nodes = self.cluster_coordinator.identify_surviving_nodes().await

if surviving_nodes.len() < self.minimum_nodes_for_recovery() {

return Err(DisasterRecoveryError::InsufficientSurvivingNodes {

available: surviving_nodes.len(),

required: self.minimum_nodes_for_recovery(),

});

}

// 2. 최신 백업에서 복구

let latest_backup = self.backup_manager.find_latest_valid_backup().await?;

// 3. 생존 노드들에 백업 배포

for node in &surviving_nodes {

self.restore_node_from_backup(node, &latest_backup).await?;

}

// 4. 새로운 위원회 구성

let new_committee = self.form_emergency_committee(&surviving_nodes)?;
```

```
// 5. 컨센서스 재시작
self.restart_consensus_with_committee(new_committee).await?;
Ok(())
}
async fn recover_from_network_partition(&mut self) -> Result<(), DisasterReco
// 1. 분할된 네트워크 세그먼트 식별
let network_segments = self.identify_network_segments().await?;
// 2. 각 세그먼트의 상태 평가
let mut segment_states = Vec::new();
for segment in &network_segments {
let state = self.evaluate_segment_state(segment).await?;
segment_states.push((segment.clone(), state));
}
// 3. 가장 진행된 세그먼트 선택
let (primary_segment, _) = segment_states.into_iter()
.max_by_key(|(_, state)| state.last_committed_round)
.ok_or(DisasterRecoveryError::NoValidSegment)?;
// 4. 다른 세그먼트들을 주 세그먼트와 동기화
for segment in &network_segments {
if segment != &primary_segment {
self.sync_segment_with_primary(segment, &primary_segment).await?;
}
}
```

```
// 5. 네트워크 연결 복원
self.restore_network_connectivity().await?;
Ok(())
}
}
```

10.5.2 자동화된 복구 워크플로우

```
pub struct AutomatedRecoveryWorkflow {
recovery_steps: Vec<RecoveryStep>,
rollback_steps: Vec<RollbackStep>,
current_step: usize,
recovery_state: RecoveryState,
}

impl AutomatedRecoveryWorkflow {
pub async fn execute_recovery(&mut self) -> Result<(), RecoveryError> {
self.recovery_state = RecoveryState::InProgress;
for (index, step) in self.recovery_steps.iter().enumerate() {
self.current_step = index;
tracing::info!("Executing recovery step {}: {}", index + 1, step.description);
match step.execute().await {
Ok(()) => {
```

```
tracing::info!("Recovery step {} completed successfully", index + 1);
}
Err(e) => {
tracing::error!("Recovery step {} failed: {:?}", index + 1, e);
// 실패 시 롤백 수행
self.execute_rollback().await?;
self.recovery_state = RecoveryState::Failed {
failed_step: index,
error: e,
};
return Err(RecoveryError::StepFailed {
step_index: index,
step_description: step.description.clone(),
error: Box::new(e),
});
}
}
}
self.recovery_state = RecoveryState::Completed;
tracing::info!("All recovery steps completed successfully");
Ok(())
}
async fn execute_rollback(&mut self) -> Result<(), RecoveryError> {
```

```
tracing::warn!("Executing rollback from step {}", self.current_step);

// 현재 단계부터 역순으로 롤백

for  step_index  in (0..=self.current_step).rev() {

if  let Some(rollback_step) = self.rollback_steps.get(step_index) {

tracing::info!("Rolling back step {}", step_index + 1);

if  let Err(e) = rollback_step.execute().await {

tracing::error!("Rollback step {} failed: {:?}", step_index + 1, e);

// 롤백 실패는 로그만 남기고 계속 진행

}

}

}

Ok(())

}

}
```

이러한 포괄적인 장애 처리와 복구 시스템을 통해 Narwhal은 다양한 장애 상황에서도 안정적으로 동작하고 빠르게 복구할 수 있습니다.